

IT22026

```
package practicing_multithreading;
import java.util.*;

public class CarParkingSystem {
    static class RegistrarParking {
        private final int carId;

        public RegistrarParking(int carId) {
            this.carId = carId;
        }

        public int getCarId() {
            return carId;
        }

        public void process() {
            System.out.println("car " + carId + " is being parked.");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            System.out.println("car " + carId + " has been parked.");
        }
    }
}
```


IT22026

250521

```
static class parkingPool {  
    private final Queue<RegistrationParking> orderQueue = new LinkedList<>();  
    public synchronized void placeOrder(RegistrationParking order) {  
        orderQueue.add(order);  
        notifyAll();  
    }  
    public synchronized RegistrationParking takeOrder() {  
        while (orderQueue.isEmpty()) {  
            try {  
                wait();  
            } catch (InterruptedException e) {  
                Thread.currentThread().interrupt();  
                return null;  
            }  
        }  
        return orderQueue.poll();  
    }  
}
```

```
static class parkingAgent extends Thread {  
    private final parkingPool pool;  
    private final int agentId;
```



```

private final int totalCars;
private static int completeOrders = 0;

public ParkingAgent(ParkingPool pool, int agentId, int totalCars) {
    this.pool = pool;
    this.agentId = agentId;
    this.totalCars = totalCars;
}

```

```

@Override
public void run() {
    while (true) {
        Registration order = pool.takeOrder();
        if (order == null) break;
        System.out.println("Agent " + agentId + " is handling car"
            + order.getCarId());
        order.process();
        synchronized (ParkingAgent.class) {
            completeOrders++;
            if (completeOrders >= totalCars) {
                System.out.println("All cars are parked.");
                break;
            }
        }
    }
}

```

2 3 4 5 6

IT22026

```
public static void main (String[] args) {  
    parkingpool pool = new parkingpool ();  
    Scanner scanner = new Scanner(System.in);  
    System.out.print ("Enter number of cars to park: ");  
    int numberOfcars = scanner.nextInt();  
    scanner.close();  
    for (int i = 1; i <= 3; i++) {  
        new parkingagent (pool, i, numberOfcars). start();  
    }  
    for (int i = 1; i <= numberOfcars; i++) {  
        registration parking order = new registrationparking(i);  
        pool.placeOrder = new registrationparking (order);  
        try {  
            Thread.sleep(500);  
        }  
        catch (InterruptedException e) {  
            Thread.currentThread().interrupt();  
        }  
    }  
}
```